



温州肯恩大学

WENZHOU-KEAN UNIVERSITY

MGS 3001 WHS01

Python Programming for Business

Python Fundamentals

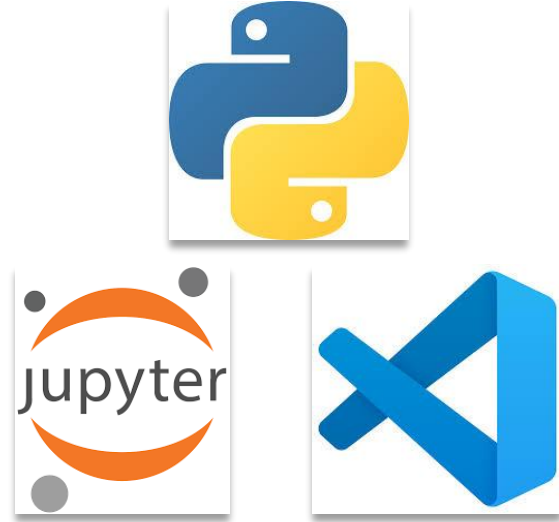
Dawei Chen

Week 2-2, 5 March 2026

Recap

- Programs are built from primitive elements.

- Numbers: 10, 3.14
- Strings: "hello"
- Operators: +, -, *, /
- etc.



- Code must follow strict syntax rules.

- | | | | |
|------------|--|---|---------------------------|
| • "hi" 5 | ← not syntactically valid | → | • "cat boy hugs" |
| • "hi" * 5 | ← syntactically valid | → | • "cat hugs boy" |
| • "hi" + 5 | ← syntactically valid, static semantic error | → | • "The table eats pizza." |

Outline

- 1) Statements and Comments
- 2) Spaces and Indentation
- 3) Input and Output
- 4) Variables and Assignment
- 5) Built-in Data Types
- 6) Python Numbers
- 7) Python Strings
- 8) Python Operators
- 9) In-class Exercise
- 10) Summary

Statements and Comments

- A Python **program** consists of a sequence of:
 - Definitions – introduce elements such as variables or functions
 - Statements – commands that instruct the computer to perform actions
- A **statement** is a single instruction executed by Python.
- **Comments** are used to explain code and ignored during execution.
 - Facilitate collaboration
 - Improve documentation
 - Enhance code maintainability

Comments starts with a #

```
#This is a comment
#written in
#more than just one line
print("Hello, Python!") #This is a comment
```

[1] ✓ 0.0s

... Hello, Python!

Unassigned strings as comments

```
"""
This is a comment
written in
more than just one line
"""

print("Hello, Python!")
```

[2] ✓ 0.0s

... Hello, Python!

Why triple quotes?
Single or double quotes?

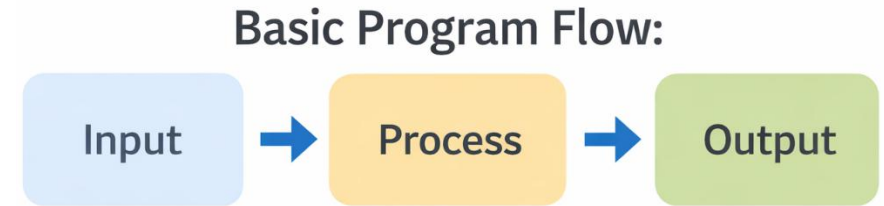
Spaces and Indentation

- Spaces
 - Improve readability of code
 - Commonly used around operators and after commas
 - Standard practice: **consistent** spacing
- Indentation
 - Leading spaces at the beginning of a line
 - Used to define code blocks in Python
 - Standard practice: **4 spaces** per indentation level

```
revenue = price * quantity
revenue=price*quantity
revenue = price*quantity
```

```
if revenue > 10000:
    print("High sales")
```

Input and Output



- Input – Getting Data into a Program
 - Read or import external data/files (e.g., CSV, databases, APIs)
 - Use `input()` to collect information interactively from the user
 - The value returned by `input()` is a string by default
 - Convert to the needed type when necessary (e.g., `int()`, `float()`)
- Output – Displaying Results
 - Use `print()` to display information on the screen
 - In Jupyter Notebook, output appears directly below the code cell after execution
 - Commonly used to show results, messages, summaries, or debugging information

Variables and Assignment

- `=`: Assignment operator — assigns a value to a variable
 - `==`: Equality operator — checks whether two values are equal
-
- Variables store data values in a program.
(Why we need?)
 - Key Concepts
 - A variable is created when a value is assigned.
 - [Naming rules for variables](#)
 - [Assigning multiple values](#)
 - [Local vs. global variables](#)
- $$x = 5$$
- ```
x = 5
x = 10
```
- Variable Assignment
    - 5 is the data value.
    - This statement assigns (binds) the value 5 to the variable x.
    - After execution, x refers to 5.
  - Reassignment
    - Variables can be updated with a new value.

Python executes code sequentially, one line at a time.

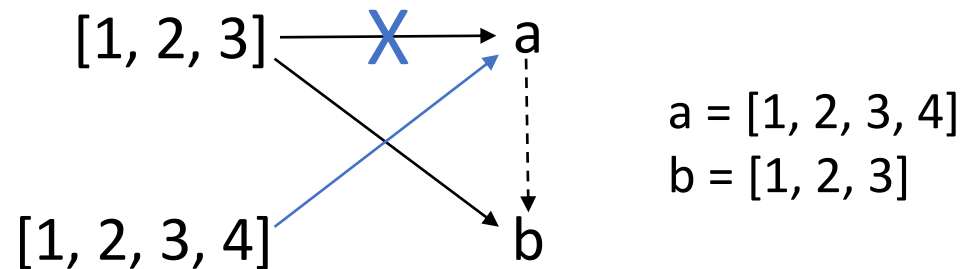
# Check Your Understanding

`append()` method in Python is used to add a single item to the end of list. This method *modifies the original list* and *does not return a new list*.

Case 1:

```
a = [1, 2, 3]
b = a
a = [1, 2, 3, 4]
```

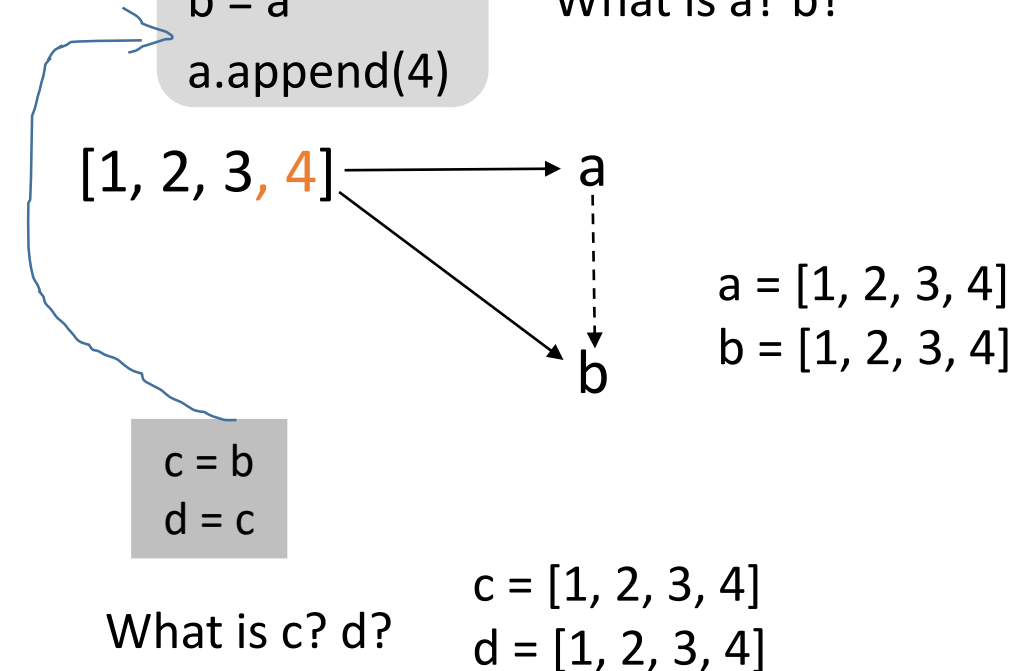
What is a? b?



Case 2:

```
a = [1, 2, 3]
b = a
a.append(4)
```

What is a? b?





# Built-in Data Types

Different data types,  
different purposes, use  
cases, properties, operations

Set, check,  
converse data type

| Data Type |                              | Example                                      |
|-----------|------------------------------|----------------------------------------------|
| Text      | str                          | "Hello, Python!"                             |
| Numeric   | int, float, complex          | 20, 20.5, 1j                                 |
| Sequence  | list, tuple, range           | [1, 2, 3], ("a", "b"), range(6)              |
| Mapping   | dict                         | {"name": "John", "age": 36}                  |
| Set       | set, frozenset               | {"name", "John", 36}                         |
| Boolean   | bool                         | True, False                                  |
| Binary    | bytes, bytearray, memoryview | b"Hello", bytearray(5), memoryview(bytes(5)) |
| None      | NoneType                     | None                                         |

# Python Numbers

- Numeric Types in Python

- Int (integer) – whole numbers without decimals (e.g., `10`, `-5` )
- float (floating-point) – numbers with decimals (e.g., `3.14`, `-0.25` )
- complex – numbers with real and imaginary parts (e.g., `3 + 2j` )

- Scientific Notation

- Floats can use scientific notation with `e` to represent powers of 10
- Example: `1.5e3 = 1500`

- Type Conversion

- Convert numeric types using built-in functions
  - `int()` → convert to integer
  - `float()` → convert to float
  - `complex()` → convert to complex number

# Python Strings

Different colors indicate different elements.

```
'This line is ignored.'
"This line is ignored."
print("Hello, Python!")
[3] ✓ 0.0s
... Hello, Python!
```

```
'''
This is a comment
written in
more than just one line
'''
print("Hello, Python!")
[5] ✓ 0.0s
... Hello, Python!
```

```
"This is
a comment"
print("Hello, Python!")
[4] ✗ 0.0s
... Cell In[4], line 1
 "This is
 ^
SyntaxError: unterminated string literal (detected at line 1)
```

```
'Hello'
"Hello"
"I'm learning Python"
'He said "Hello"'
'I'm learning Python'
```

- Strings can be defined with single quotes ' ' or double quotes " "
- **Unassigned** strings as comments
- 'xx' and "xx": only support **single-line** strings
- """xxx""" or """"xxx""": support **multi-line** strings

# Python Strings

- A string is a sequence (array) of Unicode characters used to represent text.
  - Working with strings
    - Indexing & slicing (access part of a string), looping through characters
    - Change case: upper / lower
    - Remove whitespace
    - Escape characters: `\'`, `\\`, `\n`, `\t`, `\b`
    - Replace, split, concatenate, format strings
  - String processing tools
    - Built-in [string methods](#)
    - [Regular expressions](#) for pattern matching and text extraction
    - Text mining and natural language analysis (e.g., sentiment analysis, unstructured → structured)
- “Hello, Python!”      length

0      -1

**“Hello, Python!”**      length?

# Python Operators

## Operator Precedence

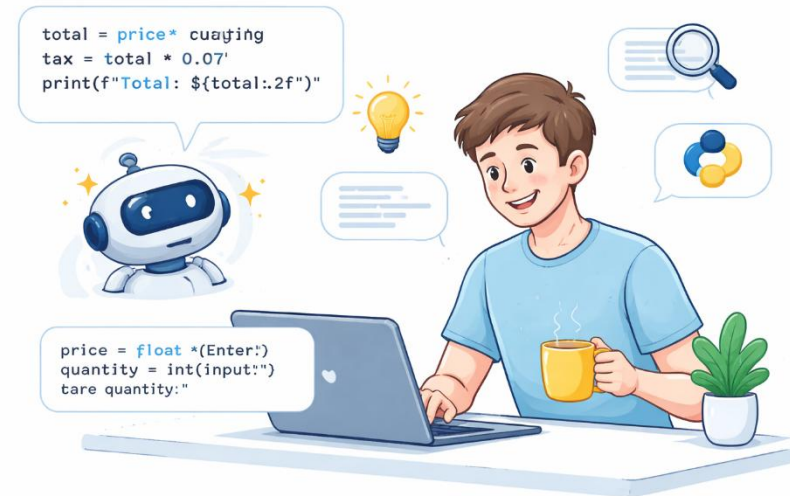
| Category             | Purpose                                              | Examples                    |
|----------------------|------------------------------------------------------|-----------------------------|
| Arithmetic Operators | Perform mathematical calculations                    | +, -, *, /, %, **, //       |
| Assignment Operators | Assign values to variables                           | =, +=, -=, *=, /=, % =, **= |
| Comparison Operators | Compare two values                                   | ==, !=, >, <, >=, <=        |
| Logical Operators    | Combine conditional expressions                      | and, or, not                |
| Identity Operators   | Check whether two variables refer to the same object | is, is not                  |
| Membership Operators | Test whether a value exists in a sequence            | in, not in                  |
| Bitwise Operators    | Perform operations on binary numbers                 | &,  , ^, ~, <<, >>          |

# In-Class Exercise

**Think:** How could this program be improved?

**Task:** Use AI tools to write a Python program that performs the following steps:

- Step 1 — Input. Ask the user to enter:
  - Product name
  - Price per item
  - Quantity purchased
- Step 2 — Process. Calculate:
  - $\text{revenue} = \text{price} * \text{quantity}$
  - $\text{tax} = \text{revenue} * 0.07$
  - $\text{total} = \text{revenue} + \text{tax}$
- Step 3 — Output. Display a formatted summary:
  - Product:
  - Price:
  - Quantity:
  - Revenue:
  - Tax:
  - Total:



*Generated by ChatGPT5.2*

# Summary

You do not need to memorize everything now. Practice reading, writing, and debugging code — familiarity develops through use.

- Python programs are built from statements and definitions.
- Comments help document and explain code.
- Spaces and indentation structure Python programs.
- Variables store data values.
- Python provides several built-in data types: number, string, sequence, etc.
- Operators perform calculations and comparisons.
- Next Class: Python data structure – list, tuple, set, dictionary, range
- **Assignment 1, DDL – 15 March, 23:59**, Beijing Time.

# Q&A

CBPM B214

[dchen@kean.edu](mailto:dchen@kean.edu)

Tue & Thu 11:15 – 12:30 | Wed 14:00 – 16:30